S. Nirmal
192511249
SOLT - D
27

SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO2: Configure IP addressing schemes, subnetting, and basic routing techniques using simulation tool, for efficient network design and topology. *(BL3)*

Assessment Tool Weightage: 30%
Software:Cisco Packet Tracer,Wireshark

Code of Conduct:

I S. Nirmal (192511249) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S. Nirmal

Criteria	Weightage	Q1 10marks	Q2 10marks	Q3 10marks	Q4 10marks	Q5
Problem Identification	20%	2	2	1	1	2
Debugging	25%	1	1	1	1	1
Optimization	20%	1	1	1	1	1
Analysis	20%	1	1	1	1	1
Code Quality	15%	1	1	1	1	1

28/40

Annexure A

Debugging & Optimisation Task

1. A network administrator has configured a workstation with the following settings:

- **IP Address:** 192.168.10.65
- **Subnet Mask:** 255.255.255.192 (/26)
- **Default Gateway:** 192.168.10.1

However, the workstation is unable to communicate with other systems configured in the subnet **192.168.10.0/26**.

Task

Debug the network configuration by:

- Identifying the subnet to which the configured IP address belongs.
- Determining the network address, broadcast address, and valid host range.
- Verifying whether the configured IP address and gateway are valid.
- Identifying the configuration errors.
- Optimizing the configuration by assigning an appropriate IP address and default gateway.
- Calculating the total number of usable host addresses after correction.

2. Two devices are configured with the following IPv6 addresses:

- Device A: **2001:db8::ff00:42:8329/64**
- Device B: **2001:db8:0:0:1::1/64**

Although both devices are connected to the same network, communication fails.

Task

Debug the IPv6 configuration by:

- Expanding both IPv6 addresses into their complete notation.
- Determining the network prefix of each device.
- Verifying whether both devices belong to the same subnet.
- Identifying the addressing or prefix mismatch.
- Optimizing the IPv6 configuration by assigning appropriate addresses.
- Calculating the number of available host addresses within the corrected subnet.

3. An organization has allocated the network **192.168.1.0/24** into several departments using the subnet masks **/26**, **/27**, and **/28**. Some departments report insufficient IP addresses, while others have a large number of unused addresses.

Task

Analyze and optimize the subnet allocation by:

- Identifying the network address, broadcast address, and host range for each subnet.
- Calculating the number of usable and unused IP addresses.
- Identifying inefficient subnet allocations.
- Debugging the addressing plan by locating subnet allocation mistakes.
- Recommending an optimized subnetting scheme using VLSM to improve address utilization while supporting current departmental requirements.

4. Two devices are configured with the following IPv6 addresses:

- Device A: **2001:db8::ff00:42:8329/64**
- Device B: **2001:db8:0:0:1::1/64**

Although both devices are connected to the same network, communication fails.

Task

Debug the IPv6 configuration by:

- Expanding both IPv6 addresses into their complete notation.
- Determining the network prefix of each device.
- Verifying whether both devices belong to the same subnet.
- Identifying the addressing or prefix mismatch.
- Optimizing the IPv6 configuration by assigning appropriate addresses.
- Calculating the number of available host addresses within the corrected subnet.

5. A router contains the following routing table.

Destination Network	**Next Hop**
192.168.1.0/24	10.0.0.2
192.168.2.0/24	10.0.0.3
Default Route	10.0.0.1

Users report that packets destined for **192.168.2.100** are not reaching the intended network.

Task

Debug the routing configuration by:

- Identifying the matching routing entry using the longest-prefix match.
- Determining the next-hop router selected for packet forwarding.
- Verifying whether the destination IP belongs to the intended subnet.
- Identifying any routing configuration errors.
- Optimizing the routing table to improve packet forwarding efficiency.
- Identifying the route that will be used if no matching destination exists.

Answer 1: Debugging the Workstation Configuration (192.168.10.65/26, Gateway 192.168.10.1)

1. Problem Identification

The workstation is configured with IP **192.168.10.65**, mask **255.255.255.192 (/26)** and default gateway **192.168.10.1**, but it cannot reach the systems of subnet **192.168.10.0/26**. The configuration must be checked to find which subnet each address really belongs to.

2. Debugging

Step 1 – Subnet of the configured IP. The block size of /26 is 256 − 192 = 64, so the subnets are .0, .64, .128 and .192. Because 64 ≤ 65 < 128, the address belongs to **192.168.10.64/26**. Binary proof for the last octet: 65 = 01000001 AND 192 = 11000000 → 01000000 = 64.

Parameter	Workstation subnet 192.168.10.64/26	Intended subnet 192.168.10.0/26
Network address	192.168.10.64	192.168.10.0
Broadcast address	192.168.10.127	192.168.10.63
Valid host range	192.168.10.65 – 192.168.10.126	192.168.10.1 – 192.168.10.62
Usable hosts	$2^6 - 2 = 62$	$2^6 - 2 = 62$
Is 192.168.10.65 here?	Yes (first usable host)	No (outside 0 – 63)
Is 192.168.10.1 here?	No (outside 64 – 127)	Yes (first usable host)

Step 2 – Validity check. The IP address 192.168.10.65 is a legal host address (not a network or broadcast address) but of the **wrong subnet**. The gateway 192.168.10.1 is a legal host address of 192.168.10.0/26 but **not** of the workstation's own subnet, 192.168.10.64/26.

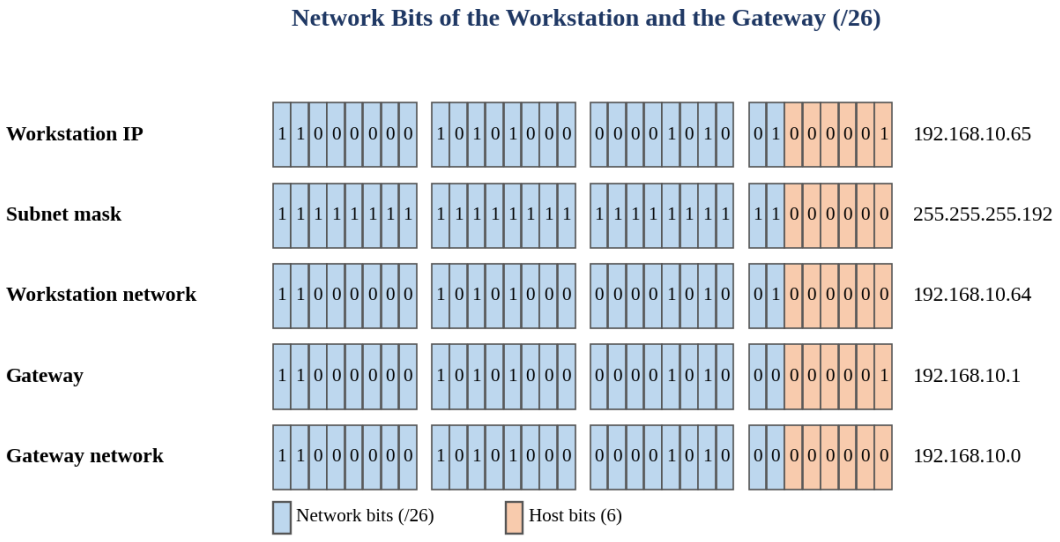


Figure 1: The workstation and the gateway have different network parts (…64 versus …0)

Step 3 – Configuration errors found.

- **Error 1 – Wrong IP subnet:** 192.168.10.65 lies in 192.168.10.64/26, not in the intended 192.168.10.0/26. The workstation therefore treats every host of .0/26 as being on a remote network.
- **Error 2 – Gateway outside the host's subnet:** a host can use a default gateway only if the gateway is on its own subnet. 192.168.10.1 gives (65 AND 192) = 64 ≠ 0, so the gateway is unreachable and is rejected.
- **Consequence:** the workstation cannot send traffic to .0/26 hosts directly (they look remote) or through the gateway (it is not reachable), so communication fails in both directions.

3. Optimization

The workstation must be moved into the intended subnet and keep a gateway that belongs to it. The corrected configuration is:

Setting	Faulty value	Corrected value
IP address	192.168.10.65	192.168.10.10
Subnet mask	255.255.255.192	255.255.255.192
Default gateway	192.168.10.1	192.168.10.1

Check: 10 AND 192 = 0, so 192.168.10.10 belongs to 192.168.10.0/26 and shares the network with the gateway 192.168.10.1. After correction the subnet offers $2^6 - 2$ = **62 usable host addresses**; since one address (192.168.10.1) is used by the gateway, 61 remain for workstations.

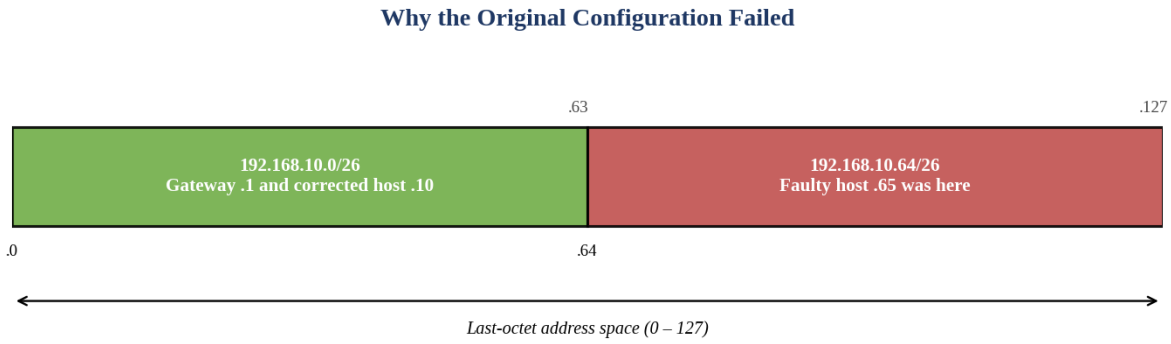


Figure 2: The faulty host and its gateway sit in two different /26 blocks; the correction moves the host into the gateway's block

4. Analysis

The fault is a subnet-placement error, not a mask error: the mask is correct, but the host address was chosen from the second /26 block. Hosts only communicate directly when their IP addresses give the same result after ANDing with the mask. An alternative fix, if the second block (192.168.10.64/26) was really intended, is to keep 192.168.10.65 but use a gateway from that block, for example 192.168.10.126; this option does not meet the stated aim of reaching 192.168.10.0/26.

Corrected configuration (Cisco Packet Tracer)

```
! ---- Packet Tracer: Router (gateway of 192.168.10.0/26) ----
Router(config)# interface gigabitEthernet 0/0
Router(config-if)# ip address 192.168.10.1 255.255.255.192  ! gateway
Router(config-if)# no shutdown
! ---- Workstation (Desktop > IP Configuration) ----
IP Address      : 192.168.10.10
Subnet Mask     : 255.255.255.192
Default Gateway : 192.168.10.1
! ---- Verification ----
PC> ipconfig  ! confirm the three values
PC> ping 192.168.10.1  ! expected: 4 replies received
```

Result and Interpretation: The workstation address 192.168.10.65 belongs to 192.168.10.64/26, while the gateway belongs to 192.168.10.0/26, which explains the failure. Assigning 192.168.10.10/26 with gateway 192.168.10.1 restores communication and leaves 62 usable addresses in the subnet (61 free for workstations).

Answer 2: Debugging the IPv6 Addresses of Device A and Device B

1. Problem Identification

Device A uses **2001:db8::ff00:42:8329/64** and Device B uses **2001:db8:0:0:1::1/64**. Both devices are on the same network but cannot communicate. Both addresses are written in compressed form, which hides the zero groups, so they must be expanded before any comparison is made.

2. Debugging

Step 1 – Expand the addresses. An IPv6 address always has eight groups. In Device A, "::" replaces 8 − 5 = 3 groups; in Device B, "::" replaces 8 − 6 = 2 groups.

```
Device A: 2001:db8::ff00:42:8329  -> 2001:0db8:0000:0000:0000:ff00:0042:8329
Device B: 2001:db8:0:0:1::1       -> 2001:0db8:0000:0000:0001:0000:0000:0001
```

Step 2 – Network prefix of each device (first 64 bits = first four groups).

Device	First four groups	Network prefix	Interface identifier
Device A	2001:0db8:0000:0000	2001:db8::/64	0000:ff00:0042:8329
Device B	2001:0db8:0000:0000	2001:db8::/64	0001:0000:0000:0001

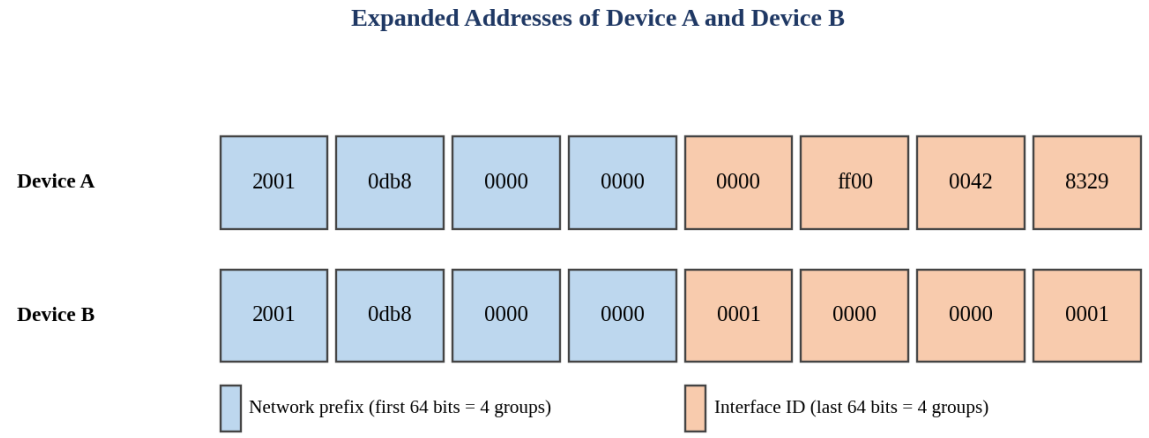


Figure 3: Both devices have the identical /64 prefix 2001:db8::/64; only the interface identifiers differ

Step 3 – Same subnet? Yes. The first 64 bits are identical, so both devices belong to the **same subnet 2001:db8::/64**. The value "1" in the fifth group of Device B is part of its interface identifier, not of the prefix. Anyone who counts this "1" as part of the network (reading Device B as 2001:db8:0:0:1::/80) will wrongly conclude that the devices are in different subnets.

Step 4 – Addressing or prefix mismatch. There is **no prefix mismatch** in the stated data. The real weaknesses are: (a) the interface identifiers are irregular (ff00:42:8329 and 1:0:0:1) and heavily compressed, which makes mistakes in configuration and reading likely; (b) the prefix 2001:db8::/32 is reserved for documentation and is not routable on a real network; and (c) the real cause of the failure must therefore be found in the actual device settings, using the checks in Figure 2.

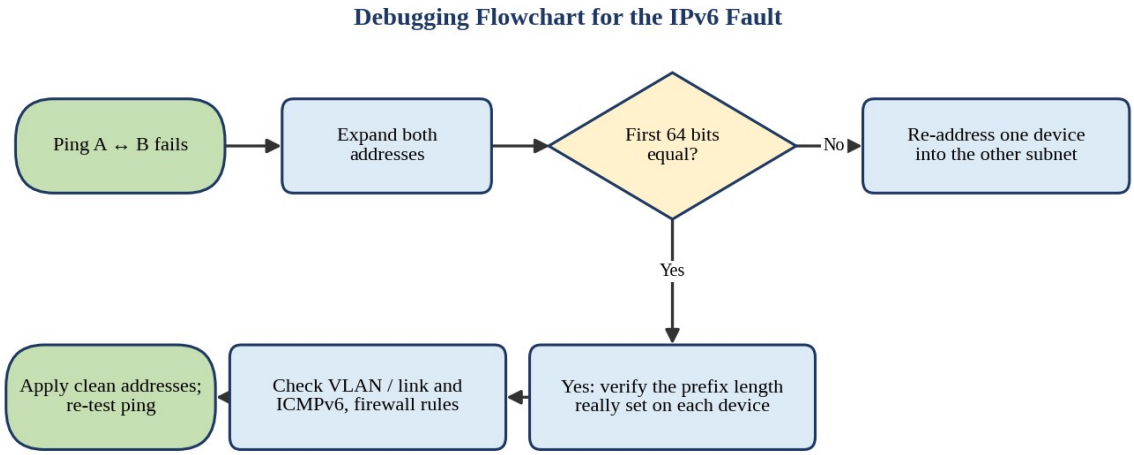


Figure 4: Systematic debugging procedure for the IPv6 communication failure

3. Optimization

Both devices are re-addressed with clear, sequential interface identifiers inside one planned /64 subnet:

Device	Corrected address	Prefix length	Subnet
Device A	2001:db8:acad:1::10	/64	2001:db8:acad:1::/64
Device B	2001:db8:acad:1::20	/64	2001:db8:acad:1::/64

Available host addresses in the corrected subnet: host bits = 128 − 64 = 64, so the count is 2^{64} = **18,446,744,073,709,551,616** addresses (IPv6 has no broadcast address; one address, the all-zero identifier, is reserved as the subnet-router anycast address). For production use, an assigned global prefix or a unique-local (fd00::/8) prefix should replace the documentation prefix.

4. Analysis

Because the stated prefixes already agree, changing the prefix would not repair a real fault; the effective actions are to confirm the configured prefix length (a /80 or /128 on one device would break on-link detection), to make sure both interfaces are in the same VLAN, and to allow ICMPv6 (Neighbor Discovery and echo). The clean addresses remove ambiguity and simplify troubleshooting.

Corrected configuration (Cisco Packet Tracer)

```
Router(config)# ipv6 unicast-routing  ! enable IPv6 routing
Router(config)# interface gigabitEthernet 0/0
Router(config-if)# ipv6 address 2001:db8:acad:1::1/64  ! gateway of the subnet
Router(config-if)# no shutdown
PC-A> ipv6 address 2001:db8:acad:1::10/64  ! Device A
PC-B> ipv6 address 2001:db8:acad:1::20/64  ! Device B
Router# show ipv6 interface brief  ! check status
PC-A> ping 2001:db8:acad:1::20  ! expected: replies received
```

Result and Interpretation: Expanded, the addresses are 2001:0db8:0000:0000:0000:ff00:0042:8329 and

2001:0db8:0000:0000:0001:0000:0000:0001. Both are in 2001:db8::/64, so the fault is not a subnet mismatch but an unclear, unplanned address layout and unverified device settings. The corrected subnet 2001:db8:acad:1::/64 holds 2^{64} = 18,446,744,073,709,551,616 addresses.

Answer 3: Analysing and Optimizing the 192.168.1.0/24 Subnet Plan with VLSM

1. Problem Identification

The organization divided 192.168.1.0/24 into departmental subnets using only the masks /26, /27 and /28. Some departments run out of addresses, while others hold large unused ranges. The original question does not list the departmental requirements, so the following are **assumed**: Engineering 100 hosts, Finance 50 hosts, Administration 25 hosts, HR 10 hosts, and a router-to-router link needing 2 addresses (187 hosts in total).

2. Debugging

Step 1 – Existing subnets (assumed allocation). For each subnet: usable hosts = $2^h - 2$; unused = usable − required.

Department	Hosts needed	Network	Host range	Broadcast	Usable	Status
Engineering	100	192.168.1.0/26	192.168.1.1 – 192.168.1.62	192.168.1.63	62	Short by 38
Finance	50	192.168.1.64/26	192.168.1.65 – 192.168.1.126	192.168.1.127	62	12 unused
Administration	25	192.168.1.128/26	192.168.1.129 – 192.168.1.190	192.168.1.191	62	37 unused
HR	10	192.168.1.192/27	192.168.1.193 – 192.168.1.222	192.168.1.223	30	20 unused
Router link	2	192.168.1.224/28	192.168.1.225 – 192.168.1.238	192.168.1.239	14	12 unused

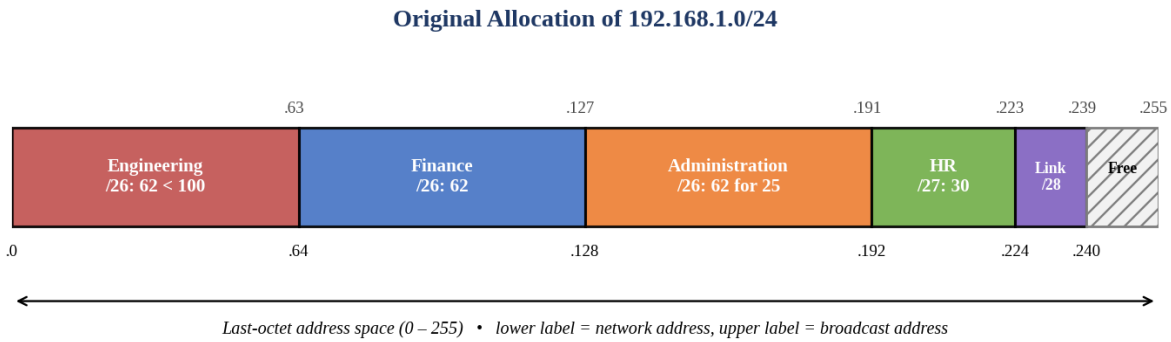


Figure 5: Original plan: equal /26 blocks ignore the real requirements

Step 2 – Inefficient allocations and mistakes.

- **Engineering (undersized):** a /26 gives only 62 hosts for a need of 100, so 38 devices have no address. It needs a /25 (126 hosts).
- **Administration (oversized):** a /26 (62 hosts) for 25 devices wastes 37 addresses; a /27 (30 hosts) is enough.
- **HR (oversized):** a /27 (30 hosts) for 10 devices wastes 20 addresses; a /28 (14 hosts) is enough.
- **Router link (oversized):** a /28 (14 hosts) for 2 addresses wastes 12 addresses; a /30 (2 hosts) is the exact size.
- **Root cause:** subnets were chosen without sorting the requirements, so equal blocks were given to departments of different sizes instead of sizing each block to its need (VLSM).

3. Optimization

Step 3 – VLSM design. Requirements are sorted from largest to smallest and each subnet receives the smallest block that satisfies it: 100 → /25 (126), 50 → /26 (62), 25 → /27 (30), 10 → /28 (14), 2 → /30 (2).

Department	Needed	Network	Host range	Broadcast	Usable	Spare
Engineering	100	192.168.1.0/25	192.168.1.1 – 192.168.1.126	192.168.1.127	126	26
Finance	50	192.168.1.128/26	192.168.1.129 – 192.168.1.190	192.168.1.191	62	12
Administration	25	192.168.1.192/27	192.168.1.193 – 192.168.1.222	192.168.1.223	30	5
HR	10	192.168.1.224/28	192.168.1.225 – 192.168.1.238	192.168.1.239	14	4
Router link	2	192.168.1.240/30	192.168.1.241 – 192.168.1.242	192.168.1.243	2	0

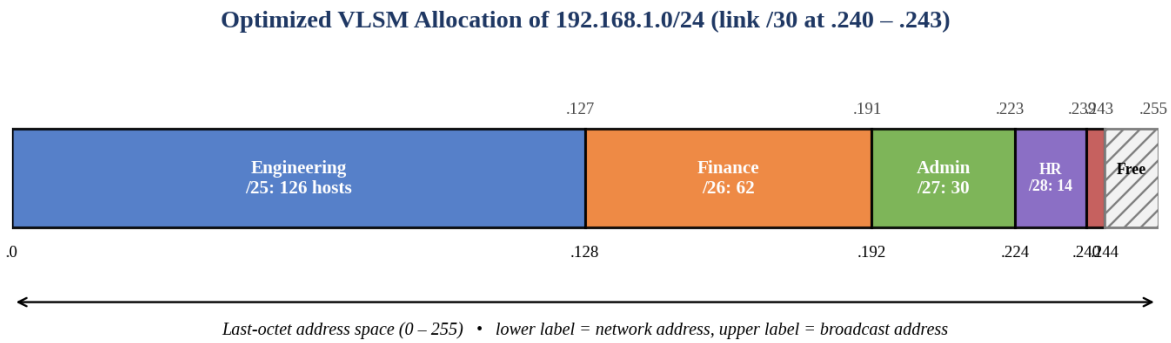


Figure 6: Optimized plan: every subnet is sized to its requirement and the free space is contiguous

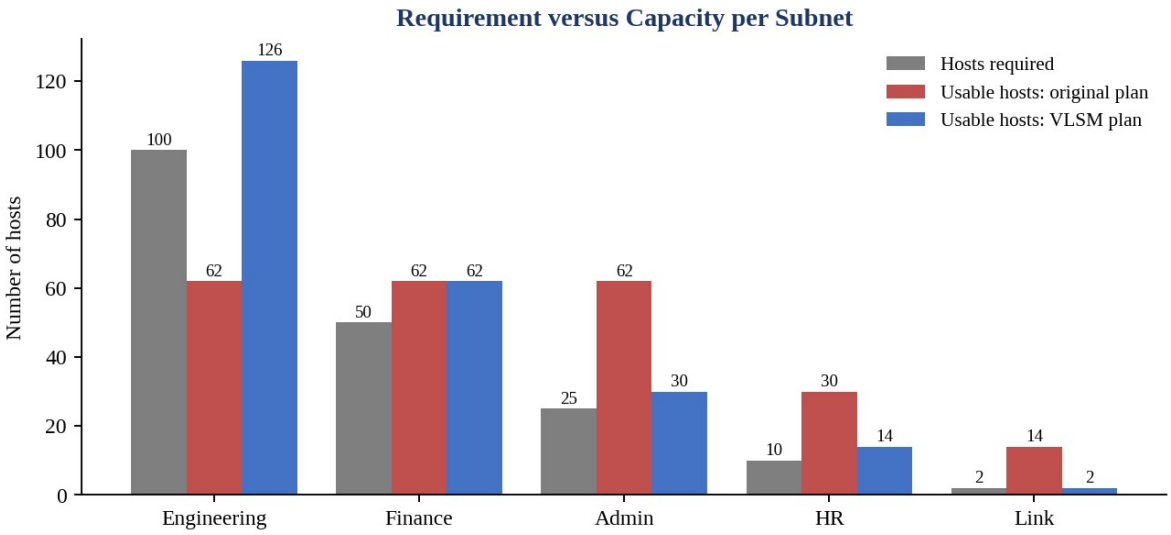


Figure 7: Original subnets fall short for Engineering and overshoot for the others; the VLSM subnets track the need

4. Analysis

Measure	Original plan	VLSM plan
Addresses allocated	240 of 256	244 of 256
Total usable host capacity	62+62+62+30+14 = 230	126+62+30+14+2 = 234

Measure	Original plan	VLSM plan
Hosts that can be served (need 187)	149 (Engineering short by 38)	187 (all requirements met)
Unused usable addresses	81	47
Capacity utilization (served ÷ capacity)	149 ÷ 230 = 64.8%	187 ÷ 234 = 79.9%
Free reserve at the end of the block	192.168.1.240 – .255 (16)	192.168.1.244 – .255 (12)

The VLSM plan satisfies every department, removes the shortage, reduces wasted usable addresses from 81 to 47 and raises utilization from about 65% to about 80%. The 12 free addresses (192.168.1.244 – .255) can still be split into a /30 and a /29 for future links.

VLSM interface configuration (Cisco Packet Tracer; every interface also needs "no shutdown")

```
Router(config)# interface g0/0
Router(config-if)# ip address 192.168.1.1 255.255.255.128  ! Engineering /25
Router(config)# interface g0/1
Router(config-if)# ip address 192.168.1.129 255.255.255.192  ! Finance /26
Router(config)# interface g0/2
Router(config-if)# ip address 192.168.1.193 255.255.255.224  ! Admin /27
Router(config)# interface g1/0
Router(config-if)# ip address 192.168.1.225 255.255.255.240  ! HR /28
Router(config)# interface s0/0/0
Router(config-if)# ip address 192.168.1.241 255.255.255.252  ! WAN link /30
```

Result and Interpretation: The mistakes were one undersized subnet (Engineering) and three oversized subnets (Administration, HR, router link). The VLSM scheme 192.168.1.0/25, .128/26, .192/27, .224/28 and .240/30 meets all assumed requirements with 244 of the 256 addresses allocated and 12 addresses held in reserve.

Answer 4: IPv6 Fault Verification Using Neighbor Discovery and Packet Capture

1. Problem Identification

This task repeats the IPv6 scenario of Answer 2: Device A has **2001:db8::ff00:42:8329/64**, Device B has **2001:db8:0:0:1::1/64**, both are on one network, and communication fails. The facts are the same, so this answer approaches the problem from the verification side, using the prefix-boundary method, Neighbor Discovery and a Wireshark capture, and proposes a different corrected plan.

2. Debugging

Step 1 – Complete notation. Count the groups written: Device A has 5 groups plus "::", so 3 zero groups are hidden; Device B has 6 groups plus "::", so 2 zero groups are hidden.

```
A = 2001:0db8:0000:0000:0000:ff00:0042:8329
B = 2001:0db8:0000:0000:0001:0000:0000:0001
```

Step 2 – Prefix boundary method. A /64 prefix equals 4 groups × 16 bits. The boundary is therefore after the fourth colon. For both devices, the part before this boundary is 2001:0db8:0000:0000, so the prefix of each is **2001:db8::/64**.

Step 3 – Subnet comparison and mismatch. The prefixes are identical, so the devices are in the same subnet. The apparent mismatch is caused by the position of the "1" in Device B: it is the first group of the interface identifier (the 5th group), not part of the prefix. A mismatch would exist only if the prefix length configured on one device were different from /64, for example /80 (the prefix of B would then become 2001:db8:0:0:1::/80 and A would be off-link).

Step 4 – Verify with Neighbor Discovery. On an IPv6 link, a device does not use ARP; it sends a Neighbor Solicitation (NS) to the solicited-node multicast address and expects a Neighbor Advertisement (NA). If the ping fails even though the prefixes agree, the capture shows exactly where the exchange stops. With the corrected plan below, the expected exchange is:

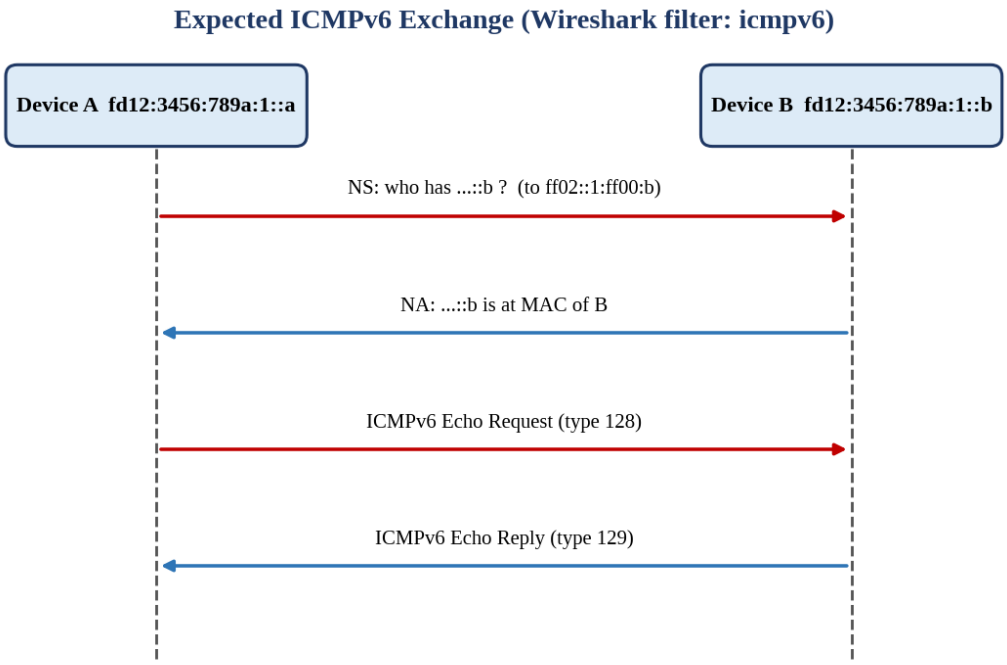


Figure 8: Neighbor Discovery followed by echo; if the NS gets no NA, the fault is at Layer 2 or in a filter

Capture result	Meaning	Action
No NS seen on the wire	Device A believes B is off-link (wrong prefix length)	Correct the prefix length to /64
NS sent, no NA returned	Different VLAN/link, or ICMPv6 blocked on B	Check switch port VLAN and firewall
NA received, no Echo Reply	Host firewall blocks echo	Permit ICMPv6 echo

3. Optimization

To avoid hard-to-read compressed addresses, a unique-local (ULA) subnet with sequential host numbers is designed:

Item	Value
Subnet	fd12:3456:789a:1::/64
Device A	fd12:3456:789a:1::a/64
Device B	fd12:3456:789a:1::b/64
Default gateway (router)	fd12:3456:789a:1::1

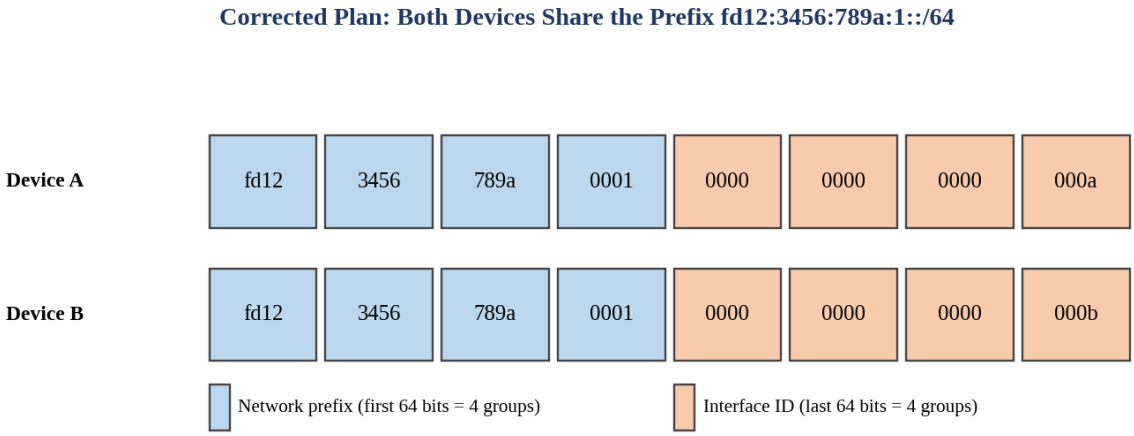


Figure 9: The corrected addresses differ only in the final group

Available hosts in the corrected subnet: 128 − 64 = 64 host bits, so 2^{64} = **18,446,744,073,709,551,616** addresses (about 1.8×10^{19}). IPv6 has no broadcast address, so only the reserved subnet-router anycast address (interface identifier ::0) is excluded in practice.

4. Analysis

Using sequential identifiers ::a and ::b makes the solicited-node address easy to predict (ff02::1:ff00:a and ff02::1:ff00:b), so the capture can be read quickly. Before the correction, the identifiers (ff00:42:8329 and 1:0:0:1) gave unpredictable solicited-node addresses and hid the structure of the network. The ULA range fd00::/8 is intended for private networks and, unlike the documentation prefix, can be used on real equipment.

Corrected configuration and verification (Cisco Packet Tracer)

```
Router(config)# ipv6 unicast-routing
Router(config)# interface gigabitEthernet 0/0
Router(config-if)# ipv6 address fd12:3456:789a:1::1/64
Router(config-if)# no shutdown
PC-A> ipv6 address fd12:3456:789a:1::a/64  ! Device A
PC-B> ipv6 address fd12:3456:789a:1::b/64  ! Device B
PC-A> ping fd12:3456:789a:1::b  ! expect 4 replies
Router# show ipv6 neighbors  ! NDP table lists both devices
```

Result and Interpretation: Both original addresses share the prefix 2001:db8::/64, so there is no subnet mismatch in the data given; the faults to fix are the unclear addressing and the unverified prefix length, link and filter settings. The corrected subnet fd12:3456:789a:1::/64 provides 18,446,744,073,709,551,616 addresses.

Answer 5: Debugging and Optimizing the Router's Routing Table

1. Problem Identification

The router holds three entries: **192.168.1.0/24 → 10.0.0.2**, **192.168.2.0/24 → 10.0.0.3** and a **default route → 10.0.0.1**. Users report that packets for **192.168.2.100** do not reach the intended network. The routing decision for this destination must be traced and every possible configuration fault examined.

2. Debugging

Step 1 – Longest-prefix match. Every entry is compared with the destination 192.168.2.100 (binary 11000000.10101000.00000010.01100100):

Entry	Prefix length	Does it match?	Reason
192.168.1.0/24	/24	No	Third octet 1 ≠ 2
192.168.2.0/24	/24	**Yes**	First 24 bits are identical
0.0.0.0/0 (default)	/0	Yes	Matches every address, lowest priority

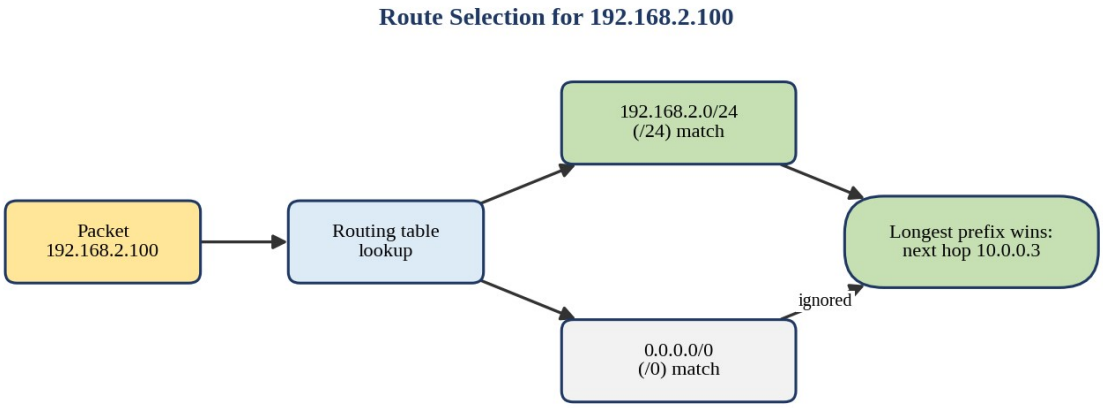


Figure 10: The /24 entry is more specific than the default route and is therefore chosen

Step 2 – Next hop selected: 10.0.0.3. **Step 3 – Destination check:** 192.168.2.0/24 has network address 192.168.2.0, broadcast 192.168.2.255 and hosts .1 – .254, so 192.168.2.100 is a valid host of the intended subnet.

Step 4 – Errors in the configuration. The lookup itself is correct, so the fault lies in what the entry points to. The following errors are possible and must be tested:

- **Wrong or unreachable next hop:** 10.0.0.3 may be the wrong router for 192.168.2.0/24 (next hops .2 and .3 may be swapped), or it may be down or not answering ARP.
- **Missing return route:** the router at 10.0.0.3 may have no route back to the source network, so replies are lost although the request arrives.
- **Segment too small:** the router, 10.0.0.1, 10.0.0.2 and 10.0.0.3 share one link. A /30 mask holds only two hosts (.1 and .2), and 10.0.0.3 would be its broadcast address, so the segment must be at least a /29.

- **Recursive lookup without an exit interface:** each route that names only a next hop needs a second lookup, which increases forwarding work and can fail if the next hop is not connected.
- **Filtering or disabled routing:** an ACL or “no ip routing” on the next-hop router would drop the traffic.

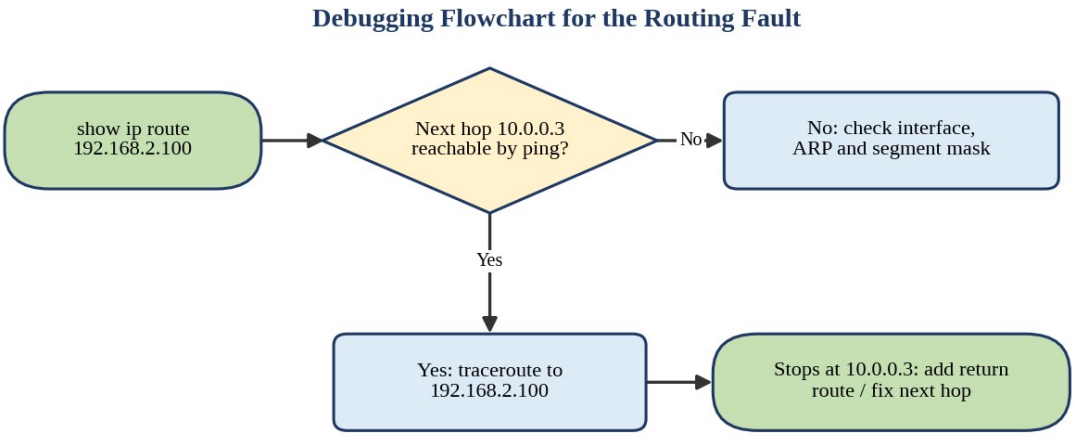


Figure 11: Systematic procedure to locate the failing hop

3. Optimization

Destination	Next hop / exit interface	Admin distance	Purpose
192.168.1.0/24	10.0.0.2 via Gi0/0	1	Network 1 (verified next hop)
192.168.2.0/24	10.0.0.3 via Gi0/0	1	Network 2 (verified next hop)
0.0.0.0/0	10.0.0.1 via Gi0/0	1	Gateway of last resort

- **Name the exit interface with the next hop** so the router avoids a recursive lookup and forwards faster.
- **Verify and, if needed, correct the next hops** with ping and traceroute before saving the configuration.
- **Use a /29 (or larger) mask** on the shared segment 10.0.0.0 so that all next hops are valid hosts.
- **Summarization:** the two customer routes cannot be merged, because they have different next hops and no single prefix (192.168.0.0/22 would also cover the unowned 192.168.0.0/24 and 192.168.3.0/24). The table already has the minimum number of entries.
- **Optional backup:** a floating static route with a higher administrative distance (for example 200) can be added through an alternative next hop for resilience.

4. Analysis

Packets for 192.168.2.100 are matched to the correct entry, so the routing-table logic is not the problem. The remaining causes are all outside the lookup: the next hop, the return path or

the segment addressing. If no entry matches a destination (for example 8.8.8.8), the router uses the **default route 0.0.0.0/0 via 10.0.0.1**, the gateway of last resort.

Corrected configuration and verification (Cisco Packet Tracer)

```
Router(config)# no ip route 192.168.2.0 255.255.255.0 10.0.0.3  ! remove old
Router(config)# ip route 192.168.2.0 255.255.255.0 g0/0 10.0.0.3
Router(config)# ip route 192.168.1.0 255.255.255.0 g0/0 10.0.0.2
Router(config)# ip route 0.0.0.0 0.0.0.0 g0/0 10.0.0.1  ! default route
Router# show ip route 192.168.2.100  ! must show /24 via 10.0.0.3
Router# ping 10.0.0.3  ! next-hop reachability
Router# traceroute 192.168.2.100  ! find the failing hop
R3(config)# ip route 0.0.0.0 0.0.0.0 10.0.0.1  ! example return route on R3
```

Result and Interpretation: The longest-prefix match for 192.168.2.100 is 192.168.2.0/24, so the packet is sent to next hop 10.0.0.3. The destination belongs to the intended subnet, so the failure comes from the next hop, its return route or the segment mask, not from route selection. The optimized table uses exit interfaces with verified next hops, and unmatched traffic follows the default route via 10.0.0.1.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation